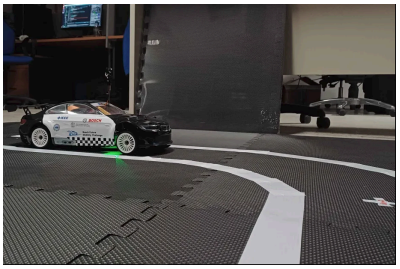
# 1 Introduction

During this phase, our team essentially took on the improvement of the previous achievements, focusing on more robust computer vision algorithms and an improved control package that utilizes the IMU's data.

# 2 Planned activities

- Set up a proper track.
- Implement a way for the car Control process to constantly handle commands and validate them.
- Make use of the onboard IMU.
- Improve the lane detection algorithm, so that it can handle curves and outliers.
- Develop code that recognizes traffic signs and estimates their pose relative to the camera.
- Implemented the traffic sign detection and started testing on the raspberry itself.

# 3 Status of planned activities



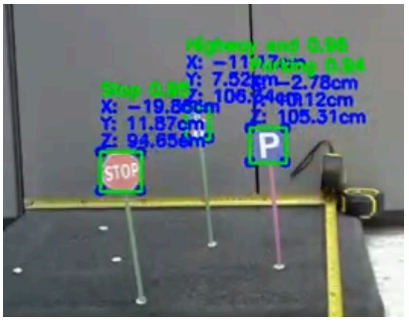
➔ **Set up a proper track.**
**Status:** Completed
**Implementation:** Using black gym floor pads and white duct tape, we made a track for testing purposes.

➔ **Car Control process improvement.**
**Status:** On Progress
**Implementation:** The Control process now can receive command messages via the Pipe system and will execute only the latest command. It also checks if the command is achievable by the car's "Ackerman" type model.
However, there was noticeable overshoot observed during testing, meaning that the PID steering calculations require improvement or maybe tuning.

➔ **IMU integration.**
**Status:** On Progress
**Implementation:** We developed code that estimates the current pose of the car using IMU data. However, the IMU readings, as printed by the Raspberry Pi on the terminal, are quite delayed. So, we are currently trying to perform the calculations on the Nucleo board instead, which proved to be more responsive to movements when printing its values on Putty.

➔ **Traffic sign recognition and pose estimation.**
**Status:** Completed
**Implementation:** We detect and localize traffic signs in 3D using a YOLO-based real-time system. Camera frames are undistorted before YOLO detects signs at 640×480 resolution. For 3D pose estimation, we use contour-based PnP calculations or fallback to bounding box-based geometric

estimation if contours fail. The system adapts to different sign types and displays detection boundaries with XYZ coordinates on the video feed. This dual approach ensures robust, real-time performance.
We are using a Yolo11n model that we have fine-tuned on a custom dataset. We printed the signs and collected images/videos in various environments, totaling about 1000 frames. Each frame was annotated using Roboflow. After augmentations the total size of the dataset grew to about 3000.
Deploying the model on the raspberry, compiled in the NCNN format, with a resolution of 640x640, we notice that the accuracy is really good and performance-wise, the algorithm achieves an average of 7-8 fps.
Re-training the model using a lower resolution, 320x320, we noticed that whilst accuracy does take a small hit, the performance boost is significant, confidently detecting in real-time at about 30-40fps, whilst only using the cpu.




➔ **Lane detection algorithm improvement.**
**Status:** On Progress
**Implementation:** The sliding window technique replaced the previous lane detection algorithm to enhance continuous line detection on turns and improve the recognition of dashed lines and intersections.

## Complications:

- After flashing the given github code for the first time in the Nucleo, it stopped responding to any commands given by both the Raspberry Pi and Putty.
  Eventually, we found out that the code responsible for receiving messages through serial was using `(void)sscanf(a,"%hhu",&l_keyValue);` and was receiving only the first byte of the value given. This was faulty in cases where the number given was larger than 1 digit (i.e. in the command “#kl:30;;\r\n” it is 2 digits, meaning 2 char variables and only the last one was received). Additionally, as the return value was not always handled, no errors were displayed and debugging was not intuitive at all.

# 4 General status of the project

The vehicle is able to maneuver through a lane, even in cases of one lane line missing (i.e. on a turn). It is also able to recognize traffic signs encountered on the track and estimate their pose.

# 5 Upcoming activities

- Estimate the car’s location using IMU.
- Tune PID hyperparameters.
- Find the aspect ratio and resolution that works best.
- Enrich the dataset, including all classes required and frames from the car.
- Connect lane detector output with control input.